



INTERNSHIP PROJECT REPORT

PhysioCare

A Full-Stack Physiotherapy Clinic Management System

Submitted as part of the Off-Campus Summer Internship Requirement

Submitted by

RAHANI GAURANGKUMAR PRGAJIBHAI

Student ID – [202512007]

MSc. [Information Technology]
July 2026

Table of Contents

1. Introduction	1
2. Context Diagram	3
3. Scope Statement	5
4. Detailed Use Cases That Have Been Implemented	7
5. Design Contribution	9
6. Programming Contribution	11
7. Tools, Technologies, APIs and Libraries Used	13
8. Testing Strategies and Reports	15
9. Innovative Contribution	17
10. Lessons Learnt	18

1. Introduction

1.1 Background

Physiotherapy clinics typically rely on a mix of manual registers, phone calls, and disconnected tools to manage patient appointments, track treatment progress, and handle payments. This creates friction for all parties involved: receptionists juggle paper diaries and cash registers, doctors have no structured way to track whether a patient is actually performing prescribed exercises at home, and patients have no visibility into their treatment plan beyond what they remember from their last visit. PhysioCare was built to address this gap by digitizing the entire clinic workflow into a single, role-aware web application.

1.2 Problem Statement

A physiotherapy clinic needs a system that lets patients book appointments and pay online without a phone call, lets front-desk staff manage walk-ins and phone bookings with immediate or deferred payment, lets doctors record consultations and prescribe structured exercise plans while getting notified immediately if a patient reports high pain or a new symptom, and gives patients a simple way to follow their day-to-day exercise schedule and log their progress. Existing generic scheduling or CRM tools do not model this specific combination of clinical workflow, structured exercise prescription, and real-time patient-reported alerts.

1.3 Objectives of the Project

- Build a single system that supports four distinct user roles — Admin, Doctor, Receptionist, and Patient — each with a dedicated dashboard and permission set.
- Support three different appointment booking modes (walk-in, phone booking, and online self-booking) with correct payment handling for each.
- Allow doctors to record consultations and build structured, date-scheduled exercise treatment plans for patients.
- Give patients a day-by-day exercise schedule they can follow and log, including pain level and comments.
- Automatically surface high-pain or new-symptom logs to the treating doctor as actionable alerts, without requiring a separate manual reporting step.
- Integrate real payment processing (Razorpay) and automated email notifications (booking confirmation, invoices, alerts) into the workflow.
- Keep the system self-maintaining through scheduled background jobs (cron) that expire unpaid bookings, mark missed sessions, and flag no-shows without manual intervention.

1.4 User Roles Overview

The system defines four roles, each backed by its own dashboard and API permissions:

Role	Description	Primary Entry Point
Admin	Super-user. Manages all user accounts and views global clinic analytics.	/admin/dashboard
Doctor	Physiotherapist. Conducts consultations, builds treatment plans, reviews patient alerts.	/doctor-dashboard
Receptionist	Front-desk staff. Books appointments, collects payments, registers new patients.	/receptionist-dashboard
Patient	End user. Books appointments, follows exercise schedule, reports concerns.	/dashboard

After login, a dedicated AuthRedirect component reads the logged-in user's role from the backend and automatically routes them to the correct dashboard — a patient never sees the doctor's screens and vice versa, both at the routing level and enforced again by backend middleware.

2. Context Diagram

The diagram below represents PhysioCare as a Level-0 Data Flow Diagram (DFD). The system is shown as a single process at the centre, surrounded by every external entity it exchanges data with — four human actors (Admin, Doctor, Receptionist, Patient) and three external services (Clerk for authentication, Razorpay for payments, and Gmail/NodeMailer for transactional email). Every entity has two distinct, separately labelled flows — one input into the system and one output from the system — rather than a single generic bidirectional arrow, so that the exact data crossing the system boundary in each direction is unambiguous.



Figure 2.1 — PhysioCare Level-0 Context Diagram

2.1 External Entities

Entity	Type	Data Sent to System	Data Received from System
Platform Admin	Human Actor	User management commands, role assignments	Global analytics, revenue & activity reports
Doctor	Human Actor	Consultation notes, treatment plan details, availability settings	Patient records, appointment list, pending alerts
Receptionist	Human Actor	Booking requests, payment collection, patient registration	Booking confirmations, invoices, patient list
Patient	Human Actor	Appointment requests, exercise logs, concern reports	Schedules, treatment plans, appointment confirmations
Clerk	External System	User identity, JWT tokens, webhook events	Auth verification requests, session validation
Razorpay Gateway	External System	Payment confirmation webhook (HMAC verified)	Payment order / link creation requests
Gmail / Nodemailer	External System	SMTP delivery status	Email send requests (welcome, invoice, confirmation, alert)

3. Scope Statement

3.1 In Scope

The following functionality is fully implemented and part of the delivered system:

- Role-based authentication and authorization for four roles (Admin, Doctor, Receptionist, Patient), backed by Clerk with a database-side role model.
- Three appointment booking flows: at-desk (walk-in), phone booking with a payment link, and patient self-booking with online checkout.
- A doctor-configurable weekly availability system with per-date overrides (leave/holiday) stored as structured JSON.
- Real-time slot computation that subtracts already-booked appointments from a doctor's configured availability.
- Full consultation and treatment-plan workflow — diagnosis, clinical notes, treatment recommendations, and a structured, date-scheduled exercise prescription.
- Automatic generation of a patient's day-by-day exercise schedule from a treatment plan's frequency rules (daily, alternate days, specific weekdays, or custom bitmask days).
- Patient-side exercise logging with pain level, completion status, comments, and optional attachments.
- An automatic alert system that surfaces high-pain or new-symptom logs to the treating doctor without any separate manual reporting step.
- Real payment processing via Razorpay (both Orders API and Payment Links API), with webhook-based confirmation and signature verification.
- Automated invoice generation with sequential, human-readable invoice numbers.
- Five transactional email templates sent via Gmail SMTP for welcome, payment link, confirmation, invoice, and urgent-alert notifications.
- Three background cron jobs that expire unpaid bookings, sweep missed exercise sessions, and flag no-show appointments — all without manual staff intervention.
- An admin console for global analytics and full user/role management.

3.2 Out of Scope

The following were deliberately not attempted in this version of the project, either due to time constraints of a solo internship build or because they fall outside the core clinic-workflow problem being solved:

- Native mobile applications (the system is a responsive web application only).
- Video consultation / telehealth calling functionality.
- Multi-clinic / multi-branch support — the system is designed for a single clinic instance.
- Insurance claim processing or integration with insurance providers.

- Automated unit/integration test suites (testing was performed manually and via Postman, detailed in Section 8).
- Advanced analytics/BI dashboards beyond the admin overview stats already implemented.
- Formal UI/UX design phase using external design tools — visual design decisions were made directly during frontend development.

4. Detailed Use Cases That Have Been Implemented

This section lists the concrete, implemented use cases grouped by role. Each use case reflects an actual, working end-to-end flow in the system, not a planned/future feature.

4.1 Patient Use Cases

1. Register an account (self-registration via Clerk) and automatically land on the patient dashboard.
2. Book an appointment online by choosing a doctor, viewing real-time available slots, and paying via Razorpay Checkout.
3. View today's exercise schedule, start a session, and see exercise name, sets, reps, instructions and demonstration video.
4. Log a completed exercise session with sets completed, pain level (0-10) and comments.
5. Report a standalone concern (independent of a scheduled session) that is not tied to a specific exercise.
6. View appointment history and manage/cancel upcoming appointments.
7. View aggregate progress statistics (compliance, completed vs. missed sessions) on a dedicated progress page.
8. Edit personal profile details.
9. Complete a forced password change on first login when the account was created by a receptionist.

4.2 Doctor Use Cases

1. Configure a weekly availability routine (day-wise shift timings) and add date-specific overrides for leave/holidays.
2. View today's schedule and appointment queue on the doctor dashboard.
3. Start a consultation for a completed appointment — record diagnosis, clinical notes, and treatment recommendations.
4. Build a structured treatment plan: select exercises from the library, set sets/reps/sessions-per-day, choose a frequency pattern, and set a start/end date range — the system auto-generates the full day-by-day schedule.
5. Maintain a personal exercise library (create, edit, delete exercises with instructions and video links).
6. View and manage the patient list, including full appointment and treatment history per patient.
7. Review pending alerts (red-flagged patient logs) and resolve them by acknowledging or requesting an urgent booking.
8. Add an exercise to an already-active treatment plan without creating a new plan from scratch.

4.3 Receptionist Use Cases

1. Register a new patient (creates both the Clerk account and the internal profile in one step, with a temporary password emailed to the patient).
2. Book a walk-in appointment and collect payment immediately (cash/card/UPI) at the desk.
3. Book a phone appointment and send the patient a Razorpay payment link by email, with a 30-minute payment window.
4. View and manage the full appointment calendar, including blocking a slot manually (e.g. for doctor unavailability).
5. View and search the patient registry.
6. View all invoices generated by the clinic.

4.4 Admin Use Cases

1. View global clinic statistics — total appointments, revenue, and active patients.
2. Create, edit, deactivate, or delete any user account, including assigning roles.
3. View and filter the full user list by role and active/inactive status.
4. Manage general clinic settings.

4.5 System Use Cases

These use cases run without direct user interaction, triggered by time or by another action in the system:

1. Automatically cancel a pending-payment appointment and free its slot if the payment window (15–30 minutes depending on booking method) expires.
2. Automatically mark past, unattended patient_schedule entries as 'missed' every night.
3. Automatically mark past, unattended appointments as 'no-show' every night.
4. Automatically clear a patient's pending alerts when the doctor issues an updated treatment plan, since the plan being refreshed makes the old alert stale.
5. Automatically provision a database user record (JIT provisioning) if a Clerk-authenticated user somehow has no corresponding row yet, defaulting their role to patient.

5. Design Contribution

As this was a solo project, all design work — database schema, system architecture, and API contract design — was done independently. This section documents the key design decisions and their reasoning.

5.1 Database Schema Design

I designed the complete relational schema from scratch, covering 13 tables. The design follows normalized relational principles while keeping a few pragmatic denormalized fields (e.g. cached counters) where recomputing them on every read would be wasteful. The core entity relationships are:

- users — a single identity table for all four roles, linked to Clerk via `clerk_id`, avoiding duplicate identity tables per role.
- patient_profiles and doctor_profiles — 1:1 extension tables holding role-specific data, keeping the core users table lean.
- appointments → consultations → treatment_plans → treatment_plan_exercises → patient_schedule — a five-level chain that models the full clinical journey from booking to a fully scheduled exercise plan.
- exercise_logs — designed to serve two purposes with one table: logging a scheduled session (linked to a treatment_plan_exercise) and logging a standalone concern (with that link left null), avoiding a duplicate 'concerns' table.
- invoices and payments — separated deliberately so an invoice (a record of what is owed) is distinct from a payment (a record of money actually received), which matters for methods like at-desk cash vs. online Razorpay payment.

A key schema-level design decision was adding a partial unique index (`unique_active_booking`) on (`doctor_id`, `appointment_date`, `start_time`) that only applies WHERE `status NOT IN ('cancelled', 'no_show')`. This pushes the double-booking prevention rule down to the database level instead of relying purely on application logic, which is more reliable under concurrent requests.

5.2 Availability System Design

Rather than creating a rigid table of fixed weekly slots, I designed the doctor's availability as a single JSONB document (`availability_rules`) with two parts: a `weekly_routine` (recurring day-of-week shifts) and `specific_dates` (one-off overrides for holidays or extra availability). This keeps a doctor's entire schedule configuration as one atomic, easy-to-update object, and the slot-generation logic simply checks `specific_dates` first and falls back to `weekly_routine` — giving override behaviour without needing a separate 'exceptions' table.

5.3 Alert System Design

Instead of creating a dedicated alerts table that would need to be kept in sync with `exercise_logs`, I designed alerts as a derived view over `exercise_logs` — any log where `doctor_reviewed` is false and either `pain_level` is 8 or higher or `issue_type` is 'new_symptom' is automatically an alert. This removes an entire class of synchronization bugs (an alert table going out of sync with the underlying log) at the cost of a slightly more complex read query, which I considered the right trade-off since alerts are read far less often than logs are written.

5.4 System Architecture

I designed the system as a classic three-tier architecture: a React single-page frontend, an Express REST API backend, and a PostgreSQL database, with three external services (Clerk, Razorpay, Gmail SMTP) integrated at the backend boundary only — the frontend never talks to these services directly, which keeps API keys and webhook secrets off the client and gives a single place to verify every external event (webhook signatures) before it touches application data. This boundary decision is reflected directly in the context diagram in Section 2.

6. Programming Contribution

Since this is a solo project, I wrote the entire codebase both frontend and backend myself, end to end, from schema to UI.

6.1 Backend Development

Built the complete Express.js REST API, structured around the schema above, covering the following route groups:

Route Group	Endpoints	Core Responsibility
/api/auth	2	Current-user profile and self-update
/api/patients	8	Patient data, dashboard aggregation, progress
/api/doctors	3	Doctor listing and profile
/api/availability	5	Weekly routine, overrides, slot computation
/api/appointments	8	All three booking flows, status, cancellation
/api/consultations	2	Consultation record creation and retrieval
/api/treatment-plans	6	Plan + exercise assignment CRUD
/api/exercises	4	Exercise library CRUD
/api/exercise-logs	5	Session logging, schedule, history, progress
/api/alerts	2	Pending alerts and resolution
/api/payments	3	Invoices and the Razorpay webhook handler
/api/receptionists	2	Patient account creation by staff
/api/admin	6	User and system management
/api/webhooks	1	Clerk user-lifecycle webhook
/api/clinic	1	Public clinic info

- Implemented `requireAuth` and `requireRole` middleware layered on top of Clerk's `clerkMiddleware()`, so every route is protected by both authentication and role checks.
- Wrote the JIT (Just-In-Time) user provisioning fallback so an authenticated Clerk user with no local database row yet is still handled gracefully instead of failing.
- Implemented the Razorpay webhook handler end to end — HMAC-SHA256 signature verification, transactional creation of invoice + payment rows, and appointment status update, all inside a single database transaction.
- Implemented the `generateSchedule()` algorithm that expands a treatment plan exercise's frequency rule (daily / alternate days / specific weekdays / custom bitmask) into individual dated rows in `patient_schedule`.
- Implemented `autoResolvePatientAlerts()` so that creating or updating a treatment plan automatically clears a patient's stale pending alerts.

- Wrote all three cron jobs (node-cron) — payment expiry sweep (every minute), missed-schedule sweep, and no-show sweep (both daily).
- Wrote the Clerk webhook handler with Svix signature verification to sync new user registrations into the local users table.

6.2 Frontend Development

Built the complete React 18 (Vite) frontend, including four full role-based dashboards and roughly 25 routed pages, using React Router v6 for navigation and Clerk's SignedIn/SignedOut guards for route protection.

- Built the patient-facing flow end to end: dashboard, appointment booking with live slot selection and embedded Razorpay Checkout, the exercise session player, and the progress page.
- Built the doctor-facing flow: dashboard with the red alert section, the multi-tab TreatmentPlanBuilder, the exercise library manager, and the availability settings screen.
- Built the receptionist-facing flow: booking modals for at-desk and phone booking, the payment collection modal, and the patient registry.
- Built the admin console: global stats, user management table with role/status filters, and clinic settings.
- Built the shared component layer (UnifiedHeader, Footer, AuthRedirect, toast notifications via react-hot-toast) reused consistently across all four dashboards.
- Wrote all API integration using the native fetch API (no external HTTP library), attaching the Clerk-issued JWT bearer token on every authenticated request.

7. Tools, Technologies, APIs and Libraries Used

7.1 Frontend

Layer	Technology
Framework	React 18 (Vite)
Routing	React Router v6
Auth UI	Clerk React (ClerkProvider, SignedIn / SignedOut guards)
Styling	Tailwind CSS (JIT mode) + vanilla CSS modules per page
HTTP Client	Native fetch API
Notifications	react-hot-toast
Icons	Lucide React
Fonts	Google Fonts (Inter, system fallbacks)

7.2 Backend

Layer	Technology
Runtime	Node.js (ESM modules)
Framework	Express.js
Authentication	Clerk Express (clerkMiddleware, getAuth)
ORM	Drizzle ORM
Database	PostgreSQL (Neon / self-hosted)
Payments	Razorpay (Orders API + Payment Links API)
Email	Nodemailer via Gmail SMTP (App Password)
Scheduled Jobs	node-cron
Webhook Verification	Svix (Clerk) + HMAC-SHA256 (Razorpay)

7.3 External Services / APIs

Service	Purpose
Clerk	Authentication, user identity, session/JWT management
Razorpay	Online payment processing (Orders + Payment Links)
Gmail SMTP	Transactional email delivery
Svix	Webhook signature verification for Clerk events

7.4 Development & Testing Tools

- Postman — used to manually exercise every backend API route and simulate different roles/auth tokens.
- Git — version control throughout development.
- VS Code — primary development environment.
- Browser DevTools — manual frontend debugging, network inspection, and responsive layout checks.

8. Testing Strategies and Reports

As a solo developer working within internship timelines, testing was carried out manually rather than through an automated test suite. Testing was done at two levels: API-level testing using Postman, and full end-to-end manual testing through the actual UI, including deliberate edge-case checks.

8.1 API Testing via Postman

Every route listed in Section 6.1 was manually exercised in Postman before being wired into the frontend. This included:

- Verifying correct success responses and status codes for valid requests under each applicable role.
- Verifying that protected routes correctly reject requests with no token, an invalid token, or a token belonging to a role not permitted for that route (e.g. a patient token hitting a doctor-only route).
- Verifying request validation — missing required fields, malformed dates, and invalid enum values (e.g. an invalid appointment status).
- Manually replaying the Razorpay webhook payload structure with correct and incorrect HMAC signatures to confirm the signature check actually rejects tampered payloads.

8.2 Manual Functional Testing

Each user-facing flow described in Section 4 was manually walked through in the browser, across all four roles, to confirm correct behaviour end to end — for example, booking an appointment as a patient and then confirming it appeared correctly on the receptionist calendar and the doctor's schedule, or completing a consultation and confirming the resulting treatment plan showed up correctly on the patient's daily schedule.

8.3 Edge Case Testing

Beyond the happy path, the following edge cases were specifically and manually tested:

- Double-booking prevention: attempting to book the same doctor at the same date/time twice, confirming the database-level partial unique index correctly blocks the second booking.
- Payment expiry: creating a phone/self-booking appointment and deliberately letting the payment window lapse, confirming the cron job correctly cancels it and frees the slot.
- Alert boundary values: logging exercises with `pain_level` exactly at 7 (should not alert) and exactly at 8 (should alert), to confirm the threshold condition behaves correctly.

- Standalone concern submission with `issue_type = 'new_symptom'` vs. other issue types, to confirm only the correct types surface as alerts.
- Auto-resolution: confirming that issuing a new treatment plan for a patient with existing pending alerts correctly bulk-clears those alerts.
- Account deactivation: deactivating a user from the admin panel and confirming that user is immediately blocked (403) from further API access even with a still-valid session.
- Availability overrides: setting a `specific_dates` override to an empty array and confirming the doctor shows as fully unavailable on that date despite a normally-active weekly routine.
- Schedule generation across all five frequency types (daily, alternate days, Mon/Wed/Fri, Tue/Thu/Sat, and custom bitmask days), checking the generated `patient_schedule` rows match the expected dates exactly.
- Re-running plan/schedule generation to confirm `onConflictDoNothing()` correctly prevents duplicate schedule rows instead of erroring or duplicating.

8.4 Summary of Testing Outcomes

All core flows across the four roles were verified to work correctly under both normal and edge-case conditions described above. Issues found during manual and Postman testing (mainly around missing role checks on a couple of early routes, and an initial version of the schedule generator not correctly handling the custom-day bitmask) were identified and fixed before being considered complete.

9. Innovative Contribution

While the overall system is a fairly standard clinic-management domain, a few specific design and implementation choices go beyond the obvious approach and are worth calling out as the more thoughtful parts of the build:

9.1 Derived Alerts Instead of a Separate Alerts Table

Rather than maintaining a dedicated alerts table that a real system would have to keep painstakingly in sync with the underlying exercise logs (insert an alert row, then remember to delete/update it when resolved, then worry about it drifting out of sync), alerts are computed on read directly from exercise_logs using a simple boolean condition. This removes an entire category of consistency bugs by construction rather than by discipline.

9.2 Three-Mode Unified Booking Architecture

Walk-in, phone, and online self-booking are three genuinely different real-world workflows with different payment timing (immediate vs. deferred vs. checkout), yet all three converge into the same appointments table and the same Razorpay webhook handler for payment confirmation. This avoids duplicating appointment logic three times while still respecting that each channel has a different UX and urgency.

9.3 Auto-Resolution of Alerts on Plan Update

When a doctor updates a patient's treatment plan, the system automatically clears that patient's pending alerts (with a clear resolution_note explaining why) instead of leaving them to pile up. This was a deliberate choice to prevent alert fatigue for doctors: an alert about pain during an exercise that's no longer even part of the patient's active plan is not useful.

9.4 JSONB-Based Flexible Availability Model

The availability_rules design lets a doctor's entire schedule recurring weekly shifts plus any number of one-off date overrides live in a single JSON document instead of dozens of rows across multiple tables, while still supporting correct override precedence at read time.

9.5 Defense-in-Depth on Webhooks

Both external webhook sources (Clerk and Razorpay) are independently signature-verified using different mechanisms appropriate to each provider (Svix headers vs. HMAC-SHA256), and both use express.raw() ahead of express.json() specifically to preserve the exact raw body bytes needed for signature verification: a detail that is easy to get wrong and silently break signature checks.

10. Lessons Learnt

Building PhysioCare solo, end to end, was as much a lesson in project discipline as it was in technical skill. Some honest takeaways:

10.1 Schema Design Decisions Are Expensive to Change Later

Early in the project I under-designed a couple of tables (particularly around how exercise logs and standalone concerns would relate) and had to migrate and rework them once the alert system's requirements became clearer. This taught me to spend more time up front thinking through how a table will be queried later, not just how it will be written to.

10.2 Webhook-Based Flows Need to Be Treated as Untrusted by Default

Getting the Razorpay and Clerk webhook signature verification correct — including the raw-body-before-JSON-parsing ordering — took more debugging than I expected. It made clear why 'just trust the callback' is not an acceptable approach for anything involving money, and why signature verification has to be one of the first things built, not a late addition.

10.3 Background Jobs Are Easy to Forget Until Something Goes Stale

The need for cron jobs (payment expiry, missed sessions, no-shows) only became obvious once I started manually testing what happens when a patient simply never pays or never shows up. It taught me to think about the 'nothing happens' failure path for every flow, not just the success path.

10.4 A Solo Full-Stack Build Requires Constant Context Switching

Moving between database schema, backend API design, and frontend UI on the same day, every day, was harder to manage than I expected. I learned to work in more deliberate vertical slices — finish one flow completely (DB → API → UI) before starting the next — rather than building all backend routes first and all frontend pages after, which had been my first instinct and led to a lot of rework.

10.5 Manual Testing Has Real Limits

Testing everything manually and via Postman caught most issues, but I also noticed myself re-testing the same happy paths repeatedly while edge cases got tested less consistently. This is the clearest gap I'm aware of in how I built this project, and the next thing I would invest in if I continued working on it is a proper automated test suite, especially around the payment and scheduling logic.

10.6 Overall

This project significantly improved my understanding of building a real, role-based, multi-actor system rather than a single-user CRUD app — particularly around designing for concurrent access (double-booking prevention), external service trust boundaries (webhooks), and keeping derived state (like alerts) correct without duplicating data. As an average-level developer going in, this was the most complete system I have built independently, and the gaps I identified above (formal testing, more careful upfront schema planning) are the areas I most want to improve for future projects.